

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_65bdc027-8a2\agent-a034ddc218cfea37b.jsonl`
- SHA-256: `a043fdd7509249312a79cb64ada078cb2e890cba7359baf5083cad1ad23cabad`
- Source modified: `2026-07-06T22:51:27+00:00`
- Imported at: `2026-07-08T16:00:06+00:00`
- project: `wf_65bdc027-8a2`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T22:48:37.058Z)

Reviewing GitHub PR #53 "feat(policy): add F4 URL reputation stage" (branch feat/f4-url-reputation -> main). ADR 0012 F4.
+357/-3, 3 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr53 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f4-url-reputation:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr53/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. (The 'regex' runtime dep is already installed in the venv.)

WHAT F4 DOES (policy.py):

- UrlReputationStage (Tier 3) under security_checks.blocked_domains. extract_caption_urls(text) uses a stdlib re regex _URL_RE.finditer
 - over content_item.link_url + content_item.text (NO network I/O). normalize_domain(): urlsplit -> hostname -> strip []/trailing-dot/casefold
 - > ip_address() for IP literals -> host.encode('idna') -> strip leading 'www.' -> _valid_domain_name (labels regex, len<=253, has dot).
- parse_domain_reputation_config: normalizes each blocked_domain; invalid ones go to invalid_domains (skipped, valid stay active); dedups.
- _domain_matches_blocked(domain, blocked) = domain==blocked or domain.endswith('.'+blocked) [subdomain match].
- Fail-open on invalid config (pass_ + warning + invalid_domain_reputation_config=True). Stage order now: topic -> url -> caption via evaluate_pipeline.

VERIFIED by main reviewer: gate green (533 passed @ 94.87%); subdomain match is correct (notbad.example NOT blocked by bad.example);
_URL_RE is ReDoS-safe (adversarial inputs up to 56K chars all <0.03s ? mandatory '.' separators + bounded {0,61} prevent catastrophic backtracking);
_URL_RE is DEVELOPER-controlled/fixed (not user-supplied like F3 patterns). Feature is dormant (no command sets blocked_domains).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.

Severity: high (real bug/regression/exploitable), medium (real evasion/latent-DoS/design), low (acceptable-v1-limitation/hygiene). Distinguish false-NEGATIVE (evasion of a blocklist ? lets bad through, matches ADR 'v1 metadata' framing) from false-POSITIVE (wrongly blocks a legit domain ? worse, breaks delivery).

DIMENSION: domain normalization & URL extraction correctness / evasion (policy.py normalize_domain, extract_caption_urls, _valid_domain_name).

Investigate ? prioritize FALSE-POSITIVES (wrongly blocking a legit domain) as higher severity than false-negatives (evasion):

- FALSE-POSITIVE risk: can a legit URL/domain ever normalize to a blocked domain incorrectly? e.g. does 'evilbad.example' or 'bad.example.attacker.com' match blocked 'bad.example'? (endswith('.bad.example') semantics). Does userinfo

'http://bad.example@good.com/' extract good.com or bad.example
(urlsplit.hostname)? Does a path/query containing a blocked domain
('good.com/redirect?u=bad.example') wrongly match?
- FALSE-NEGATIVE / evasion (v1-limitation, likely low): IDN homographs; host.encode('idna') is IDNA2003 with gaps ? a domain that fails idna -> None ->
not blocked; IP-as-decimal/octal/hex (http://3627734734) ; url-encoded host; trailing-dot already handled; uppercase already handled. Which evasions
work, and are they acceptable-v1 (ADR says metadata-only) or worth noting?
- normalize_domain robustness: does it ever RAISE (uncaught) on weird input rather than returning None? (ip_address, encode('idna'), urlsplit on junk).
Run a fuzz-ish battery of odd inputs and confirm it always returns str|None, never throws (a throw in the stage would be caught fail-open, but confirm).
- _URL_RE: any adversarial input causing >1s (try a few more shapes than the reviewer did). Any correctness gaps (misses obvious URLs / over-extracts)?
Run repros. Report findings with honest severity.

2. assistant (2026-07-06T22:48:40.066Z)

I'll review PR #53's domain normalization and URL extraction logic. Let me start by reading the code.

3. user (2026-07-06T22:48:42.187Z)

(Bash completed with no output)

4. assistant (2026-07-06T22:48:45.090Z)

Working directory reset. Let me use the checked-out worktree directly.

5. user (2026-07-06T22:48:45.546Z)

C:\Users\avidu\Projects\wt-pr53\src\tg_video_filter\policy.py

6. user (2026-07-06T22:48:47.641Z)

```
1      """Policy pipeline primitives for ADR 0012.
2
3      The policy layer is deliberately small and framework-free in F1: stages receive
4      an immutable context, return a structured result, and the evaluator stops on
5      the first failure. Later Track F phases add concrete stages.
6      """
7
8      from __future__ import annotations
9
10     import json
11     import logging
12     import re
13     from dataclasses import dataclass, field
14     from ipaddress import ip_address
15     from typing import Protocol
16     from urllib.parse import urlsplit
17
18     import regex
19
20     from tg_video_filter.models import ContentItem, Policy, Source
21
22     log = logging.getLogger(__name__)
23
24     _MAX_CAPTION_SCAN_CHARS = 4096
25     _MAX_CAPTION_TERMS = 100
26     _MAX_CAPTION_PATTERNS = 25
27     _CAPTION_REGEX_TIMEOUT_SECONDS = 0.01
28     _URL_RE = re.compile(
29         r"""(?ix)
```

```

30         (?<![@\\w.-])
31         (
32             (?:https?://|www\\.)[^\\s<>"]]+
33             |
34             (?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\\.)+[a-z]{2,}
35             (?:/[^\\s<>"]*)?
36         )
37         ""
38     )
39     _TRAILING_URL_PUNCTUATION = ".,!?;:"
40
41
42     @dataclass(frozen=True)
43     class PolicyContext:
44         """Immutable snapshot available to a policy stage."""
45
46         content_item: ContentItem
47         source: Source
48         policy: Policy
49         fetcher: object | None = None
50         budget: object | None = None
51
52
53     @dataclass(frozen=True)
54     class StageResult:
55         """Outcome of one policy stage."""
56
57         passed: bool
58         reason: str
59         stage_name: str
60         metadata: dict[str, object] = field(default_factory=dict)
61
62         @staticmethod
63         def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
64             return StageResult(
65                 passed=True,
66                 reason=reason,
67                 stage_name=stage_name,
68                 metadata=metadata,
69             )
70
71         @staticmethod
72         def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
73             return StageResult(
74                 passed=False,
75                 reason=reason,
76                 stage_name=stage_name,
77                 metadata=metadata,
78             )
79
80
81     class PolicyStage(Protocol):
82         """A single stateless gate in the policy pipeline."""
83
84         async def evaluate(self, ctx: PolicyContext) -> StageResult:
85             """Evaluate ``ctx`` and return a stage result."""
86             ...
87
88
89     @dataclass(frozen=True)
90     class TopicPreferences:
91         """Parsed ADR 0012 topic preference config."""
92
93         include: tuple[str, ...] = ()
94         exclude: tuple[str, ...] = ()

```

```

95         mode: str = "any"
96
97     @property
98     def enabled(self) -> bool:
99         return bool(self.include or self.exclude)
100
101
102     @dataclass(frozen=True)
103     class CaptionSafetyConfig:
104         """Parsed ADR 0012 caption safety config."""
105
106         unsafe_terms: tuple[str, ...] = ()
107         unsafe_patterns: tuple[str, ...] = ()
108         invalid_patterns: tuple[str, ...] = ()
109         ignored_term_count: int = 0
110         ignored_pattern_count: int = 0
111
112     @property
113     def enabled(self) -> bool:
114         return bool(self.unsafe_terms or self.unsafe_patterns)
115
116     @property
117     def has_config_warnings(self) -> bool:
118         return bool(self.invalid_patterns or self.ignored_term_count or
119 self.ignored_pattern_count)
120
121     @dataclass(frozen=True)
122     class DomainReputationConfig:
123         """Parsed ADR 0012 URL/domain reputation config."""
124
125         blocked_domains: tuple[str, ...] = ()
126         invalid_domains: tuple[str, ...] = ()
127
128     @property
129     def enabled(self) -> bool:
130         return bool(self.blocked_domains)
131
132
133     @dataclass(frozen=True)
134     class ExtractedUrl:
135         """URL discovered from captured text with its normalized host domain."""
136
137         original_url: str
138         domain: str
139
140
141 def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
142     """Parse ``topics`` from a raw policy config object."""
143     raw_topics = config.get("topics")
144     if raw_topics is None:
145         return TopicPreferences()
146     if not isinstance(raw_topics, dict):
147         raise ValueError("topics must be a JSON object")
148     include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
149     exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
150     if not include and not exclude:
151         return TopicPreferences()
152     mode = raw_topics.get("mode", "any")
153     if mode not in ("any", "all"):
154         raise ValueError("topics.mode must be 'any' or 'all'")
155     return TopicPreferences(
156         include=include,
157         exclude=exclude,
158         mode=str(mode),

```

```

159     )
160
161
162 def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
163     """Parse caption safety settings from ``security_checks``.
164
165     Expected shape:
166     ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
167     """
168     raw_security = config.get("security_checks")
169     if raw_security is None:
170         return CaptionSafetyConfig()
171     if not isinstance(raw_security, dict):
172         raise ValueError("security_checks must be a JSON object")
173
174     raw_terms = _string_tuple(
175         raw_security.get("unsafe_caption_terms"),
176         field_name="security_checks.unsafe_caption_terms",
177     )
178     raw_patterns = _string_tuple(
179         raw_security.get("unsafe_caption_patterns"),
180         field_name="security_checks.unsafe_caption_patterns",
181     )
182
183     unsafe_terms = raw_terms[:_MAX_CAPTION_TERMS]
184     unsafe_patterns: list[str] = []
185     invalid_patterns: list[str] = []
186     for pattern in raw_patterns[:_MAX_CAPTION_PATTERNS]:
187         try:
188             regex.compile(pattern, regex.IGNORECASE)
189         except regex.error:
190             invalid_patterns.append(pattern)
191         else:
192             unsafe_patterns.append(pattern)
193
194     return CaptionSafetyConfig(
195         unsafe_terms=unsafe_terms,
196         unsafe_patterns=tuple(unsafe_patterns),
197         invalid_patterns=tuple(invalid_patterns),
198         ignored_term_count=max(0, len(raw_terms) - _MAX_CAPTION_TERMS),
199         ignored_pattern_count=max(0, len(raw_patterns) - _MAX_CAPTION_PATTERNS),
200     )
201
202
203 def parse_domain_reputation_config(config: dict[str, object]) -> DomainReputationConfig:
204     """Parse URL/domain reputation settings from ``security_checks``.
205
206     raw_security = config.get("security_checks")
207     if raw_security is None:
208         return DomainReputationConfig()
209     if not isinstance(raw_security, dict):
210         raise ValueError("security_checks must be a JSON object")
211
212     raw_domains = _string_tuple(
213         raw_security.get("blocked_domains"),
214         field_name="security_checks.blocked_domains",
215     )
216     blocked_domains: list[str] = []
217     invalid_domains: list[str] = []
218     seen: set[str] = set()
219     for raw_domain in raw_domains:
220         domain = normalize_domain(raw_domain)
221         if domain is None:
222             invalid_domains.append(raw_domain)
223             continue
224         if domain not in seen:

```

```

224         blocked_domains.append(domain)
225         seen.add(domain)
226     return DomainReputationConfig(
227         blocked_domains=tuple(blocked_domains),
228         invalid_domains=tuple(invalid_domains),
229     )
230
231
232 def load_topic_preferences(policy: Policy) -> TopicPreferences:
233     """Load topic preferences from ``policy.config_json``."""
234     decoded = json.loads(policy.config_json)
235     if not isinstance(decoded, dict):
236         raise ValueError("policy config_json must decode to a JSON object")
237     return parse_topic_preferences(decoded)
238
239
240 def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
241     """Load caption safety settings from ``policy.config_json``."""
242     decoded = json.loads(policy.config_json)
243     if not isinstance(decoded, dict):
244         raise ValueError("policy config_json must decode to a JSON object")
245     return parse_caption_safety_config(decoded)
246
247
248 def load_domain_reputation_config(policy: Policy) -> DomainReputationConfig:
249     """Load URL/domain reputation settings from ``policy.config_json``."""
250     decoded = json.loads(policy.config_json)
251     if not isinstance(decoded, dict):
252         raise ValueError("policy config_json must decode to a JSON object")
253     return parse_domain_reputation_config(decoded)
254
255
256 def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
257     if value is None:
258         return ()
259     if not isinstance(value, list):
260         raise ValueError(f"{field_name} must be an array")
261     values: list[str] = []
262     for item in value:
263         if not isinstance(item, str):
264             raise ValueError(f"{field_name} must contain strings")
265         text = item.strip()
266         if text:
267             values.append(text)
268     return tuple(values)
269
270
271 class TopicPreferenceStage:
272     """ADR 0012 Tier 2 topic matching over source title and captured caption."""
273
274     stage_name = "topic-preferences"
275
276     async def evaluate(self, ctx: PolicyContext) -> StageResult:
277         try:
278             prefs = load_topic_preferences(ctx.policy)
279         except (json.JSONDecodeError, ValueError) as exc:
280             log.warning(
281                 "topic preferences disabled due to invalid config policy_id=%s",
282                 ctx.policy.id,
283                 exc_info=True,
284             )
285         return StageResult.pass_(
286             self.stage_name,
287             f"invalid topic config ignored: {exc}",
288             invalid_topic_config=True,

```

```

289         )
290     if not prefs.enabled:
291         return StageResult.pass_(self.stage_name, "topic preferences disabled")
292
293     haystack = _topic_haystack(ctx)
294     matched_excludes = _matched_topics(prefs.exclude, haystack)
295     if matched_excludes:
296         return StageResult.fail(
297             self.stage_name,
298             f"excluded topic matched: {matched_excludes[0]}",
299             matched_exclude_topics=list(matched_excludes),
300         )
301
302     if not prefs.include:
303         return StageResult.pass_(self.stage_name, "no include topics configured")
304
305     matched_includes = _matched_topics(prefs.include, haystack)
306     if prefs.mode == "any":
307         if matched_includes:
308             return StageResult.pass_(
309                 self.stage_name,
310                 f"matched include topic: {matched_includes[0]}",
311                 matched_include_topics=list(matched_includes),
312             )
313         return StageResult.fail(self.stage_name, "no include topics matched")
314
315     missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
316     if missing:
317         return StageResult.fail(
318             self.stage_name,
319             f"missing required topic: {missing[0]}",
320             matched_include_topics=list(matched_includes),
321             missing_include_topics=list(missing),
322         )
323     return StageResult.pass_(
324         self.stage_name,
325         "matched all include topics",
326         matched_include_topics=list(matched_includes),
327     )
328
329
330 def _topic_haystack(ctx: PolicyContext) -> str:
331     parts = [ctx.source.title or "", ctx.content_item.text or ""]
332     return "\n".join(parts).casefold()
333
334
335 def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
336     return tuple(topic for topic in topics if topic.casefold() in haystack)
337
338
339 def normalize_domain(value: str) -> str | None:
340     """Normalize a configured or extracted domain for blocklist matching."""
341     text = value.strip()
342     if not text:
343         return None
344
345     parsed = urlsplit(text if "://" in text else f"//{text}")
346     host = parsed.hostname
347     if host is None:
348         return None
349
350     host = host.strip().strip("[]").rstrip(".").casefold()
351     if not host:
352         return None

```

```

353
354     try:
355         return ip_address(host).compressed.casefold()
356     except ValueError:
357         pass
358
359     try:
360         domain = host.encode("idna").decode("ascii").casefold()
361     except UnicodeError:
362         return None
363     if domain.startswith("www."):
364         domain = domain[4:]
365     if not _valid_domain_name(domain):
366         return None
367     return domain
368
369
370 def extract_caption_urls(text: str | None) -> tuple[ExtractedUrl, ...]:
371     """Extract plain URLs from captured caption text without network I/O."""
372     if not text:
373         return ()
374
375     urls: list[ExtractedUrl] = []
376     for match in _URL_RE.finditer(text):
377         raw_url = match.group(1).rstrip(_TRAILING_URL_PUNCTUATION)
378         while raw_url.endswith("(") and raw_url.count("(") < raw_url.count(")"):
379             raw_url = raw_url[:-1]
380         domain = normalize_domain(raw_url)
381         if domain is not None:
382             urls.append(ExtractedUrl(original_url=raw_url, domain=domain))
383     return tuple(urls)
384
385
386 def _valid_domain_name(domain: str) -> bool:
387     if len(domain) > 253 or "." not in domain:
388         return False
389     labels = domain.split(".")
390     return all(
391         re.fullmatch(r"[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?", label) is not None for
392         label in labels
393     )
394
395
396 def _domain_matches_blocked(domain: str, blocked_domain: str) -> bool:
397     return domain == blocked_domain or domain.endswith(f".{blocked_domain}")
398
399
400 class UrlReputationStage:
401     """ADR 0012 Tier 3 captured-URL domain blocklist checks."""
402
403     stage_name = "url-reputation"
404
405     async def evaluate(self, ctx: PolicyContext) -> StageResult:
406         try:
407             config = load_domain_reputation_config(ctx.policy)
408         except (json.JSONDecodeError, ValueError) as exc:
409             log.warning(
410                 "domain reputation disabled due to invalid config policy_id=%s",
411                 ctx.policy.id,
412                 exc_info=True,
413             )
414             return StageResult.pass_(
415                 self.stage_name,
416                 f"invalid domain reputation config ignored: {exc}",
417                 invalid_domain_reputation_config=True,

```



```

417         )
418
419     if config.invalid_domains:
420         log.warning(
421             "domain reputation ignored invalid domains policy_id=%s
invalid_domains=%s",
422             ctx.policy.id,
423             list(config.invalid_domains),
424         )
425     if not config.enabled:
426         metadata = _url_reputation_metadata((), config)
427         if config.invalid_domains:
428             return StageResult.pass_(
429                 self.stage_name,
430                 "domain reputation has no valid blocked domains",
431                 **metadata,
432             )
433         return StageResult.pass_(self.stage_name, "domain reputation disabled")
434
435     urls = extract_caption_urls(ctx.content_item.link_url) + extract_caption_urls(
436         ctx.content_item.text
437     )
438     if not urls:
439         return StageResult.pass_(
440             self.stage_name,
441             "no links to scan",
442             **_url_reputation_metadata(urls, config),
443         )
444
445     for url in urls:
446         for blocked_domain in config.blocked_domains:
447             if _domain_matches_blocked(url.domain, blocked_domain):
448                 return StageResult.fail(
449                     self.stage_name,
450                     f"blocked domain matched: {blocked_domain}",
451                     **_url_reputation_metadata(urls, config),
452                     blocked_domain=url.domain,
453                     matched_blocklist_domain=blocked_domain,
454                     matched_url=url.original_url,
455                 )
456
457     return StageResult.pass_(
458         self.stage_name,
459         "links passed domain reputation checks",
460         **_url_reputation_metadata(urls, config),
461     )
462
463
464 def _url_reputation_metadata(
465     urls: tuple[ExtractedUrl, ...],
466     config: DomainReputationConfig,
467 ) -> dict[str, object]:
468     domains = sorted({url.domain for url in urls})
469     metadata: dict[str, object] = {
470         "link_count": len(urls),
471         "domains": domains,
472         "blocked_domain_count": len(config.blocked_domains),
473     }
474     if config.invalid_domains:
475         metadata["ignored_invalid_domains"] = list(config.invalid_domains)
476     return metadata
477
478
479 class CaptionSafetyStage:
480     """ADR 0012 Tier 3 captured-caption term and pattern scanning."""

```

```

481
482     stage_name = "caption-safety"
483
484     async def evaluate(self, ctx: PolicyContext) -> StageResult:
485         try:
486             config = load_caption_safety_config(ctx.policy)
487         except (json.JSONDecodeError, ValueError) as exc:
488             log.warning(
489                 "caption safety disabled due to invalid config policy_id=%s",
490                 ctx.policy.id,
491                 exc_info=True,
492             )
493             return StageResult.pass_(
494                 self.stage_name,
495                 f"invalid caption safety config ignored: {exc}",
496                 invalid_caption_safety_config=True,
497             )
498         if not config.enabled:
499             metadata = _caption_safety_metadata(config, caption_present=False)
500             if config.has_config_warnings:
501                 log.warning(
502                     "caption safety has no valid active checks policy_id=%s
503                     metadata=%s",
504                     ctx.policy.id,
505                     metadata,
506                 )
507                 return StageResult.pass_(
508                     self.stage_name,
509                     "caption safety has no valid active checks",
510                     **metadata,
511                 )
512             return StageResult.pass_(self.stage_name, "caption safety disabled")
513         if config.has_config_warnings:
514             log.warning(
515                 "caption safety ignored invalid or excess config policy_id=%s "
516                 "invalid_patterns=%s ignored_terms=%s ignored_patterns=%s",
517                 ctx.policy.id,
518                 list(config.invalid_patterns),
519                 config.ignored_term_count,
520                 config.ignored_pattern_count,
521             )
522
523         caption = (ctx.content_item.text or "").strip()
524         if not caption:
525             return StageResult.pass_(
526                 self.stage_name,
527                 "no caption to scan",
528                 **_caption_safety_metadata(config, caption_present=False),
529             )
530
531         scan_caption = caption[:_MAX_CAPTION_SCAN_CHARS]
532         caption_truncated = len(caption) > _MAX_CAPTION_SCAN_CHARS
533         haystack = scan_caption.casefold()
534         for term in config.unsafe_terms:
535             if term.casefold() in haystack:
536                 return StageResult.fail(
537                     self.stage_name,
538                     f"unsafe caption term matched: {term}",
539                     **_caption_safety_metadata(
540                         config,
541                         caption_present=True,
542                         caption_truncated=caption_truncated,
543                     ),
544                     match_type="term",

```

```

545         matched_caption_term=term,
546     )
547
548     timed_out_patterns: list[str] = []
549     for pattern in config.unsafe_patterns:
550         try:
551             matched_pattern = regex.search(
552                 pattern,
553                 scan_caption,
554                 flags=regex.IGNORECASE,
555                 timeout=_CAPTION_REGEX_TIMEOUT_SECONDS,
556             )
557         except TimeoutError:
558             timed_out_patterns.append(pattern)
559             log.warning(
560                 "caption safety regex timed out policy_id=%s pattern=%r",
561                 ctx.policy.id,
562                 pattern,
563             )
564             continue
565
566         if matched_pattern:
567             return StageResult.fail(
568                 self.stage_name,
569                 "unsafe caption pattern matched",
570                 **_caption_safety_metadata(
571                     config,
572                     caption_present=True,
573                     caption_truncated=caption_truncated,
574                     timed_out_patterns=tuple(timed_out_patterns),
575                 ),
576                 match_type="pattern",
577                 matched_caption_pattern=pattern,
578             )
579
580     return StageResult.pass_(
581         self.stage_name,
582         "caption passed safety checks",
583         **_caption_safety_metadata(
584             config,
585             caption_present=True,
586             caption_truncated=caption_truncated,
587             timed_out_patterns=tuple(timed_out_patterns),
588         ),
589     )
590
591
592 def _caption_safety_metadata(
593     config: CaptionSafetyConfig,
594     *,
595     caption_present: bool,
596     caption_truncated: bool = False,
597     timed_out_patterns: tuple[str, ...] = (),
598 ) -> dict[str, object]:
599     metadata: dict[str, object] = {
600         "caption_present": caption_present,
601         "checked_terms": len(config.unsafe_terms),
602         "checked_patterns": len(config.unsafe_patterns),
603     }
604     if caption_truncated:
605         metadata["caption_truncated"] = True
606         metadata["max_caption_scan_chars"] = _MAX_CAPTION_SCAN_CHARS
607     if config.invalid_patterns:
608         metadata["ignored_invalid_caption_patterns"] = list(config.invalid_patterns)
609     if config.ignored_term_count:

```

```

610         metadata["ignored_caption_term_count"] = config.ignored_term_count
611     if config.ignored_pattern_count:
612         metadata["ignored_caption_pattern_count"] = config.ignored_pattern_count
613     if timed_out_patterns:
614         metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
615     return metadata
616
617
618     async def evaluate_pipeline(
619         ctx: PolicyContext,
620         stages: list[PolicyStage],
621     ) -> tuple[bool, list[StageResult]]:
622         """Evaluate stages in order and short-circuit on the first failure."""
623         results: list[StageResult] = []
624         for stage in stages:
625             result = await stage.evaluate(ctx)
626             results.append(result)
627             if not result.passed:
628                 return False, results
629         return True, results
630
631
632     __all__ = [
633         "CaptionSafetyConfig",
634         "CaptionSafetyStage",
635         "DomainReputationConfig",
636         "ExtractedUrl",
637         "PolicyContext",
638         "PolicyStage",
639         "StageResult",
640         "TopicPreferenceStage",
641         "TopicPreferences",
642         "UrlReputationStage",
643         "evaluate_pipeline",
644         "extract_caption_urls",
645         "load_caption_safety_config",
646         "load_domain_reputation_config",
647         "load_topic_preferences",
648         "normalize_domain",
649         "parse_caption_safety_config",
650         "parse_domain_reputation_config",
651         "parse_topic_preferences",
652     ]
653

```

7. assistant (2026-07-06T22:48:50.797Z)

Now let me run a battery of fuzz/edge-case tests against the normalization and extraction logic.

8. assistant (2026-07-06T22:49:05.984Z)

```

from tg_video_filter.policy import (
    normalize_domain,
    extract_caption_urls,
    _valid_domain_name,
    _domain_matches_blocked,
)

def nd(x):
    try:
        r = normalize_domain(x)
        return repr(r)
    except Exception as e:
        return f"!!RAISED {type(e).__name__}: {e}"

```

```

print("=== normalize_domain robustness / correctness ===")
cases = [
    "bad.example",
    "www.bad.example",
    "BAD.Example",
    "bad.example.",
    "sub.bad.example",
    "http://bad.example@good.com/",      # userinfo trick
    "http://good.com/redirect?u=bad.example", # blocked in path/query
    "http://user:pass@bad.example/",
    "3627734734",                       # decimal IP
    "0x7f.0.0.1",                       # hex ip label
    "http://[:1]/",
    "http://127.0.0.1/",
    "http://256.256.256.256/",
    "évilbad.example",                  # idna
    "xn--nxasmq6b.example",
    "-bad.example",
    "bad-.example",
    "a"*300 + ".com",
    "",
    " ",
    "http://",
    "://nohost",
    "javascript:alert(1)",
    "mailto:x@bad.example",
    "ftp://bad.example/",
    "http://bad..example/",
    "http://.bad.example/",
    "bad.example:8080",
    "HTTP://WWW.BAD.EXAMPLE/",
    "? bad.example",                    # zero width
    "bad?example",                     # ideographic full stop as dot
    "?.example",
    "?bad.example",                    # fullwidth dot
    "www.www.bad.example",
    "wwwbad.example",
    "http://bad.example#frag",
]
for c in cases:
    print(f"{c!r:55} -> {nd(c)}")

print()
print("=== extract_caption_urls ===")
texts = [
    "visit http://bad.example now",
    "go to http://bad.example@good.com/ please",    # userinfo
    "http://good.com/redirect?u=bad.example",      # path
    "email me at foo@bad.example",                 # bare email
    "the file bad.example.txt is here",            # filename-ish
    "check bad.example.",
    "(see http://bad.example)",
    "text with bad?example dot",
]
for t in texts:
    print(f"{t!r}")
    for u in extract_caption_urls(t):
        print(f"    original={u.original_url!r} domain={u.domain!r}")

print()
print("=== match semantics ===")
for d, b in [
    ("evilbad.example", "bad.example"),
    ("bad.example.attacker.com", "bad.example"),
    ("bad.example", "bad.example"),

```



```

original='bad.example' domain='bad.example'
'(see http://bad.example)'
original='http://bad.example' domain='bad.example'
'text with bad?example dot'

=== match semantics ===
domain=evilbad.example          blocked=bad.example      -> False
domain=bad.example.attacker.com blocked=bad.example      -> False
domain=bad.example              blocked=bad.example      -> True
domain=sub.bad.example          blocked=bad.example      -> True
domain=notbad.example           blocked=bad.example      -> False

```

11. assistant (2026-07-06T22:49:16.247Z)

Several interesting results. Let me dig into the most significant ones. The `bad?example` (ideographic full stop) case is very notable: it normalizes to `bad.example`. This means IDNA maps the ideographic full stop U+3002 to a regular dot. Let me check the false-positive angle carefully and the `?.example` case, plus test a few more decimal-IP and homograph cases.

12. assistant (2026-07-06T22:49:32.334Z)

```

from tg_video_filter.policy import normalize_domain, extract_caption_urls,
_domain_matches_blocked

```

```

def nd(x):
    try:
        return repr(normalize_domain(x))
    except Exception as e:
        return f"!!RAISED {type(e).__name__}: {e}"

```

FALSE-POSITIVE hunt: does a legit domain ever normalize to a blocked one?

```

print("=== IDNA fullwidth/ideographic dot mapping (potential false-positive or evasion) ===")

```

If blocked list has 'bad.example', can a DIFFERENT legit string normalize to it?

```

for c in ["bad?example", "bad?example", "bad?example", "?.example"]:
    print(f"{c!r} -> {nd(c)}")

```

The reverse: a legit domain that IDNA-maps such that it collides.

```

print()
print("=== case-mapping collisions (German sharp s, Turkish, Greek) ===")
for c in ["straße.de", "STRASSE.de", "?.example", "???.example"]:
    print(f"{c!r} -> {nd(c)}")

```

```

print()
print("=== decimal/octal/hex IP evasions (false-negative) ===")

```

3627734734 = 216.58.214.14 as decimal. normalize returned None above.

```

for c in ["3627734734", "0330.0.0.1", "0xC0.0x00.0x02.0xEB", "192.0x0.2.1", "1.1", "127.1"]:
    print(f"{c!r} -> {nd(c)}")

```

```

print()
print("=== trailing dot in EXTRACTED url that is a blocked domain w/ subdomain ends with bug ===")

```

_domain_matches_blocked uses ends with('.'+blocked). Check a domain that ends with

blocked string but is a legit different registrant, e.g. blocked='example.com'

```

for d,b in [("myexample.com", "example.com"), ("notexample.com", "example.com"),
            ("evil-example.com", "example.com"), ("evilexample.com", "example.com")]:
    print(f"domain={d:22} blocked={b:15} -> {_domain_matches_blocked(d,b)}")

```

```

print()
print("=== single trailing char: blocked='co' would match many (config side) ===")

```

config normalizes 'co' -> needs a dot, so 'co' alone invalid. But 'a.co'?

```
print("normalize('co') ->", nd("co"))
print("normalize('a.co') ->", nd("a.co"))
```

if someone blocks 'a.co', does 'data.co' match? endswith('a.co') -> no. good.

```
print("_domain_matches_blocked('data.co','a.co') ->", _domain_matches_blocked("data.co","a.co"))
```

13. user (2026-07-06T22:49:32.365Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\fuzz2.py (file state is current in your context ? no need to Read it back)

14. user (2026-07-06T22:49:37.317Z)

```
=== IDNA fullwidth/ideographic dot mapping (potential false-positive or evasion) ===
```

```
'bad?example' -> 'bad.example'
'bad?example' -> 'bad.example'
'bad?example' -> 'bad.example'
'??.example' -> 'a.example'
```

```
=== case-mapping collisions (German sharp s, Turkish, Greek) ===
```

```
'straße.de' -> 'strasse.de'
'STRASSE.de' -> 'strasse.de'
'??.example' -> 'xn--i-9bb.example'
'???.example' -> 'bad.example'
```

```
=== decimal/octal/hex IP evasions (false-negative) ===
```

```
'3627734734' -> None
'0330.0.0.1' -> '0330.0.0.1'
'0xC0.0x00.0x02.0xEB' -> '0xc0.0x00.0x02.0xeb'
'192.0x0.2.1' -> '192.0x0.2.1'
'1.1' -> '1.1'
'127.1' -> '127.1'
```

```
=== trailing dot in EXTRACTED url that is a blocked domain w/ subdomain ends with bug ===
```

```
domain=myexample.com          blocked=example.com      -> False
domain=notexample.com         blocked=example.com      -> False
domain=evil-example.com       blocked=example.com      -> False
domain=evilexample.com        blocked=example.com      -> False
```

```
=== single trailing char: blocked='co' would match many (config side) ===
```

```
normalize('co') -> None
normalize('a.co') -> 'a.co'
_domain_matches_blocked('data.co','a.co') -> False
```

15. assistant (2026-07-06T22:49:43.095Z)

Now I have a clear picture. Key finding: `straße.de` normalizes to `strasse.de` ? this is the IDNA2003 vs IDNA2008 divergence, and it's a real false-positive AND false-negative issue. Let me confirm the severity of the `straße.de` case, which is the strongest finding: it produces a genuine **collision** where a legitimate distinct registered domain and a blocked domain map to the same value.

Let me verify: in IDNA2008/UTS46 transitional=false, `straße.de` maps to `xn--strae-oqa.de` (a real, different registrable domain than `strasse.de`). Python's `.encode('idna')` uses IDNA2003 which maps ß to ss. Let me confirm and also check the `www.` stripping edge case which could be a false-positive.

16. assistant (2026-07-06T22:49:56.100Z)

```
from tg_video_filter.policy import normalize_domain as nd
```

CONFIRM the ß collision is a real-world false positive.

strasse.de and straÙe.de are DIFFERENT registrable domains in reality (IDNA2008/UTS46)
Python idna (IDNA2003) collapses ß -> ss, so straÙe.de -> strasse.de.

```
print("straÙe.de ->", nd("straÙe.de")) # legit German domain xn--strae-oqa.de in reality
print("strasse.de ->", nd("strasse.de"))
print("COLLISION:", nd("straÙe.de") == nd("strasse.de"))
print()
```

www stripping: a legit domain literally named 'www.tld'? A registered domain like 'www.hackplayers.de' is not a legit domain
Also: after idna, a domain 'www.<blocked>' subdomain gets www stripped -> could FALSE-POSITIVE be a subdomain, but more interesting: does stripping 'www.' from a 2-label domain over-

```
print("www.com ->", nd("www.com")) # 'www.com' is a real registered domain!
print("wwwx.com ->", nd("wwwx.com"))
print()
```

Config side: if operator blocks 'www.com', it normalizes to 'com' (invalid, no... 'com' has no tld)

```
from tg_video_filter.policy import parse_domain_reputation_config as parse
cfg = parse({"security_checks":{"blocked_domains":["www.com","strasse.de","evil.example"]}})
print("blocked:", cfg.blocked_domains)
print("invalid:", cfg.invalid_domains)
print()
```

So if operator intends to block the real site www.com, it collapses to 'com' -> invalid -> blocked
And if operator blocks 'strasse.de', then legit 'straÙe.de' visitor URL gets FALSE-POSITIVE

Extraction over-match: does a version/number token get treated as a domain?

```
from tg_video_filter.policy import extract_caption_urls as ex
for t in ["upgrade to v2.3 today", "ratio 16.9 aspect", "file report.docx here",
         "see 1.2.3.4 address", "meet at 3.30pm", "price is 9.99usd"]:
    ds = [(u.original_url,u.domain) for u in ex(t)]
    print(f"{t!r:35} -> {ds}")
```

17. user (2026-07-06T22:49:56.125Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\fuzz3.py (file state is current in your context ? no need to Read it back)

18. user (2026-07-06T22:50:01.724Z)

```
straÙe.de -> strasse.de
strasse.de -> strasse.de
COLLISION: True
```

```
www.com -> None
wwwx.com -> wwwx.com
```

```
blocked: ('strasse.de', 'evil.example')
invalid: ('www.com',)
```

```
'upgrade to v2.3 today' -> []
'ratio 16.9 aspect' -> []
'file report.docx here' -> [('report.docx', 'report.docx')]
'see 1.2.3.4 address' -> []
'meet at 3.30pm' -> []
'price is 9.99usd' -> []
```

19. assistant (2026-07-06T22:50:14.485Z)

Confirmed several findings. The two most important:

1. ****`www.com` false-negative + the `www.`-strip applied to the config side unconditionally**** ?
`www.com` is a real registered domain, but normalization strips `www.` leaving `com`, which is invalid ? dropped. More importantly the strip is applied to already-normalized 2-label hosts, so an operator can never block a domain whose registrable name legitimately begins with `www.` And on the extraction side, any subdomain `www.<host>` gets collapsed, meaning a blocklist entry for a specific `www.` host is unmatchable. Let me verify the ordering: the www strip happens AFTER idna but the same for both config and extraction, so they're consistent ? the real defect is `www.com`-style loss. This is a low/medium false-negative.

2. ****ß / IDNA2003 collision (`straße.de` ? `strasse.de`)**** ? this is a genuine ****false-positive****: if an operator blocks `strasse.de`, a completely unrelated legitimate domain `straße.de` (real registrable domain `xn--strae-oqa.de`) gets blocked. This is the higher-severity finding per the false-positive priority.

Let me confirm the `www.` strip ordering precisely and check one more false-positive vector: the `\_valid\_domain\_name` re-uses stdlib `re` with a label regex ? confirm the idna output can contain only valid chars, and check whether `parse\_domain\_reputation\_config` strips `www` from a blocked entry like `www.bad.example` making it `bad.example` (broadening the block ? a false-positive risk).

20. assistant (2026-07-06T22:51:27.439Z)

API Error: Connection closed mid-response. The response above may be incomplete.